Feature Space Geometry and Loss Functions

Optimization Based Linear Classifiers

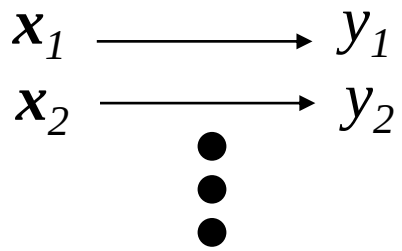
- We will discuss four linear classifiers with different objective functions:
  - **Linear regression**: MSE (mean squared error) loss
  - **Perceptron algorithm**: Perceptron loss (only misclassified samples involved)
  - **Logistic regression** (LR): Cross-entropy loss
  - **Support vector machine** (SVM): Margin between classes
- The perspective of our focus: The design of an optimization-based classifier starts with the design of its **loss function** and its properties / implications in the feature space. Then the "algorithm" is just the step of optimization with the given loss function.

Linear Regression for Binary Classification

The target outputs are set to +1 and –1 for the two classes):

$$\boldsymbol{X} = [\boldsymbol{x}_1, \boldsymbol{x}_2, \cdots, \boldsymbol{x}_N]^T$$

$$\boldsymbol{y} = [y_1, y_2, \cdots, y_N]^T$$

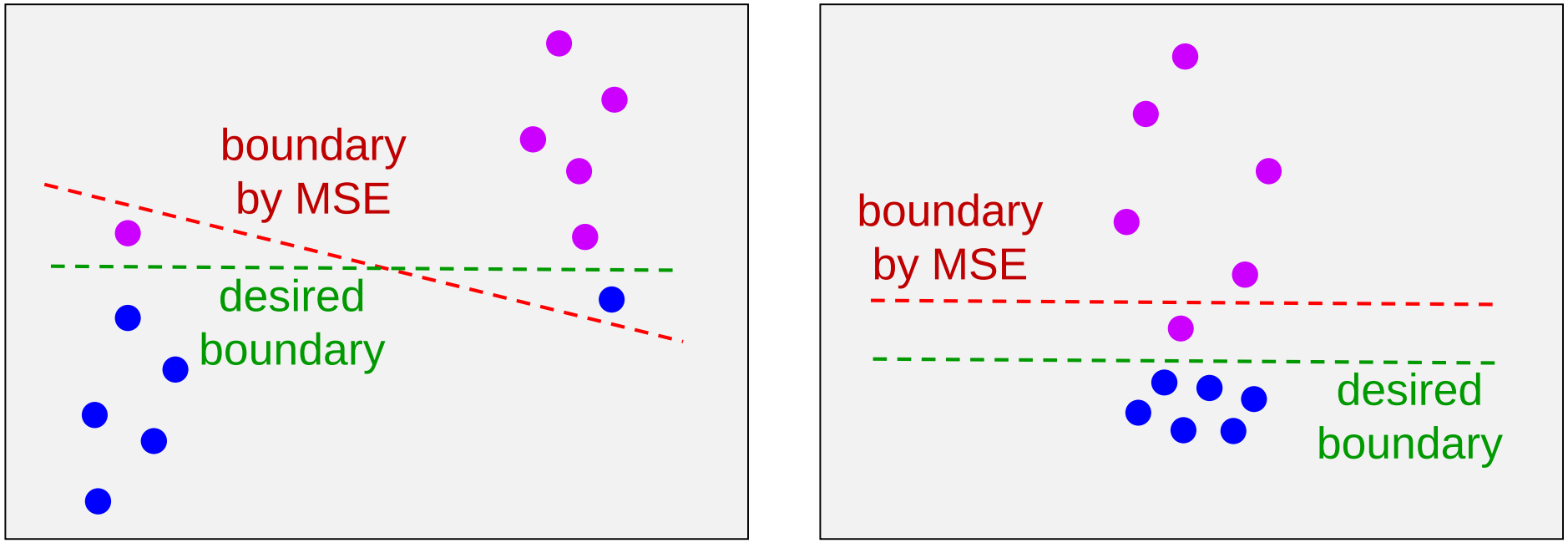


Linear regression with mean squared error (MSE) loss function:

$$J(\boldsymbol{w}) = \sum_{i=1}^{N} (y_i - \boldsymbol{x}_i^T \boldsymbol{w})^2 = \| \boldsymbol{y} - \boldsymbol{X}\boldsymbol{w} \|^2$$

Linear Regression for Binary Classification

Illustrations that minimizing MSE ≠ minimizing classification error



The MSE loss attempts to make the samples have similar distances to the decision boundary.

➔ The solution is degraded by correctly classified high-confidence outliers.

Perceptron Algorithm

- The loss function consists of contributions only from misclassified samples:

$$J(\boldsymbol{w}) = \sum_i \max\left(0, -g(\boldsymbol{x}_i) y_i\right) = \sum_i \max\left(0, -\boldsymbol{w}^T \boldsymbol{x}_i y_i\right)$$

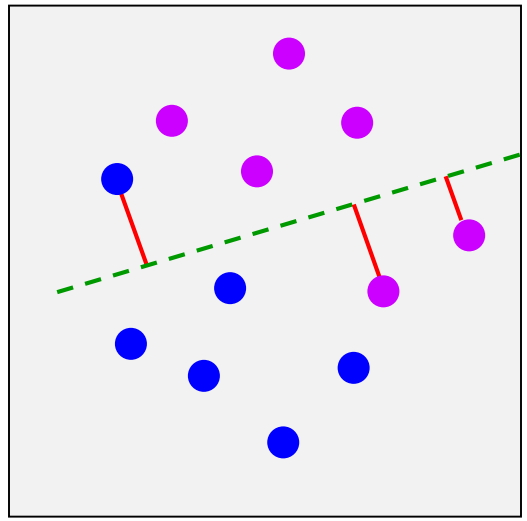
  - Distances of misclassified samples to the decision boundary in the feature space (see the illustration).

- This is an example of **hinge loss**.

- Gradient:

$$\frac{\partial J}{\partial \boldsymbol{w}} = \begin{cases} -\boldsymbol{x}_i y_i & \text{for misclassified } \boldsymbol{x}_i \\ 0 & \text{for correctly classified } \boldsymbol{x}_i \end{cases}$$

- Update equations based on subgradient-descent of $J(\boldsymbol{w})$.



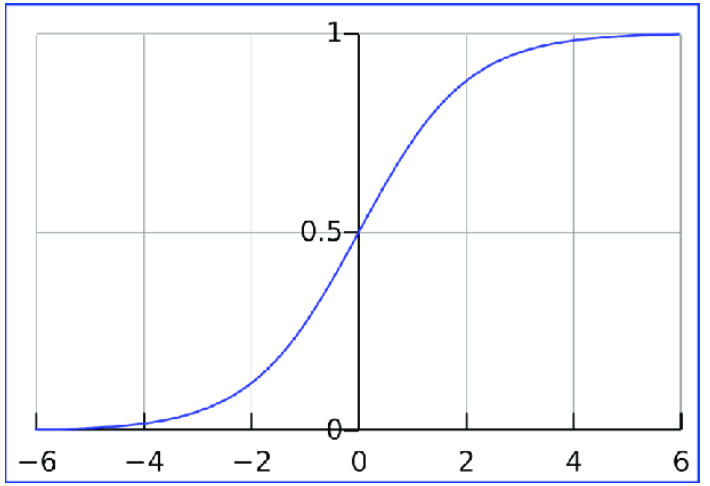
Perceptron Algorithm

- Solutions (if existent) are not unique.
- Gradients from misclassified samples do not depend on their decisions to the decision boundary ➔ Data that are not linearly separable result in infinite updates.
- Correctly classified samples have no contribution ➔ Decision boundaries may be very close to the samples even if classification is correct, reducing robustness.

Logistic Regression

- The idea: Map the sample distances to the decision boundary to a smooth continuous function that can be treated as probabilities.
- **Logistic function** is the most commonly used form:

$$f(z) = \frac{1}{1+e^{-z}}$$



  - This is a form of **sigmoid** functions.
- Parameterized for linear classification:

$$p(\boldsymbol{x}) = \frac{1}{1+e^{-\boldsymbol{w}^T \boldsymbol{x}}}$$

- Still considers global distributions of samples, not focusing on the boundary. But not as sensitive to outliers as linear regression.

Loss Function for Logistic Regression

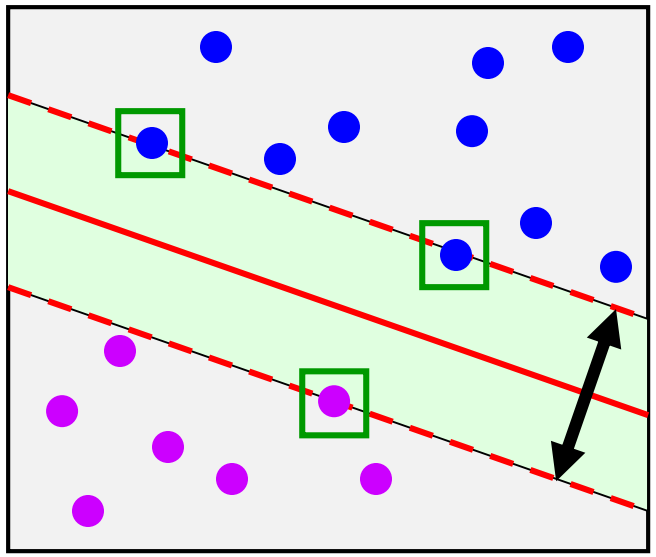
- Deriving the loss function with a probabilistic view:
  - Let the target outputs y for the two classes be 1 or 0.
  - Let the predicted outputs (probabilities in the two classes) of a sample $\boldsymbol{x}$ be p_1 and $p_0 = 1 - p_1$.
  - Likelihood: $\prod_{\boldsymbol{x}\in\omega_1} p_1(\boldsymbol{x}) \prod_{\boldsymbol{x}\in\omega_0} p_0(\boldsymbol{x})$
  - Loglikelihood: $\sum_{\boldsymbol{x}\in\omega_1} \ln\left(p_1(\boldsymbol{x})\right) + \sum_{\boldsymbol{x}\in\omega_0} \ln\left(p_0(\boldsymbol{x})\right) = \sum_{\boldsymbol{x}_i} \left[y_i \ln\left(p_1(\boldsymbol{x}_i)\right) + (1-y_i) \ln\left(p_0(\boldsymbol{x}_i)\right)\right]$
  - The negative loglikelihood is the **cross-entropy** between two distributions (target and predicted), each consisting of probabilities in two classes.
- Solve $\boldsymbol{w}$ with gradient descent by substituting the logistic form of p into this loss.

Support Vector Machine (SVM)

Idea (SVM as a linear classifier): In addition to focusing on correct classification (as in perceptron algorithm), we also want to maximize the distances of the decision boundary to the samples.

This is similar to a modification of perceptron loss (not used in our derivation here, just to show their relation) ➔

$$J(\boldsymbol{w}) = \sum_i \max\left(0, 1 - \boldsymbol{w}^T \boldsymbol{x}_i y_i\right)$$



For linearly separable cases:

The ***margin*** is twice the distance from the hyperplane to the closest sample. The ***support vectors*** are the samples that are closest to the hyperplane.

This consideration of margin allows SVM to find more robust decision boundaries than the perceptron algorithm.

Support Vector Machine (SVM)

The linear discriminant function is $g(\boldsymbol{x}) = \boldsymbol{w}^T \boldsymbol{x} + w_0$

Note that we do not use augmented vectors here.

The distance between $\boldsymbol{x}$ and the decision boundary ($g(\boldsymbol{x})=0$) is given by $\dfrac{\left|\boldsymbol{w}^T \boldsymbol{x} + w_0\right|}{\|\boldsymbol{w}\|}$

To make the values of $\boldsymbol{w}$ and w_0 unique, we add the requirement: $\left|\boldsymbol{w}^T \boldsymbol{x} + w_0\right| = 1$ if $\boldsymbol{x}$ is a support vector.

The margin is now given by $2/\|\boldsymbol{w}\|$. Maximizing the margin is the same as minimizing $\|\boldsymbol{w}\|$.

The task of finding the decision boundary in SVM is now an optimization problem:

To minimize $J = \dfrac{1}{2}\|\boldsymbol{w}\|^2$

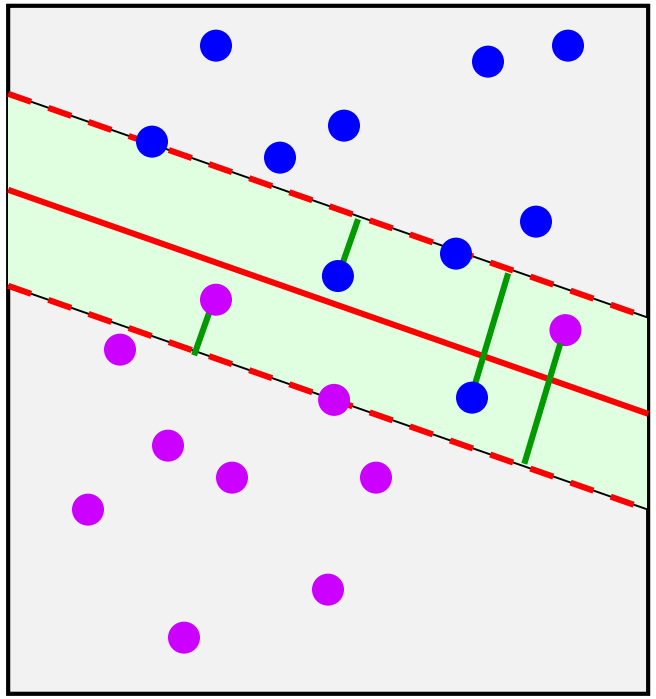
Subject to the constraints $y_i(\boldsymbol{w}^T \boldsymbol{x}_i + w_0) \geq 1, \ \forall i$

Nonseparable Cases in SVM

In most cases, the two classes are not linearly separable. This means that the previous constraints ($y_i(\boldsymbol{w}^T \boldsymbol{x}_i + w_0) \geq 1, \ \forall i$) are not valid.

Idea: We allow some degree of violating the original constrains with this modified version:

$$y_i(\boldsymbol{w}^T \boldsymbol{x}_i + w_0) \geq 1 - \xi_i \quad \text{where} \quad \xi_i \geq 0$$



Cases with $\xi_i \geq 0$ (indicated in the figure with green line segments) correspond to those $\boldsymbol{x}_i$ that are misclassified or within the "separation band".

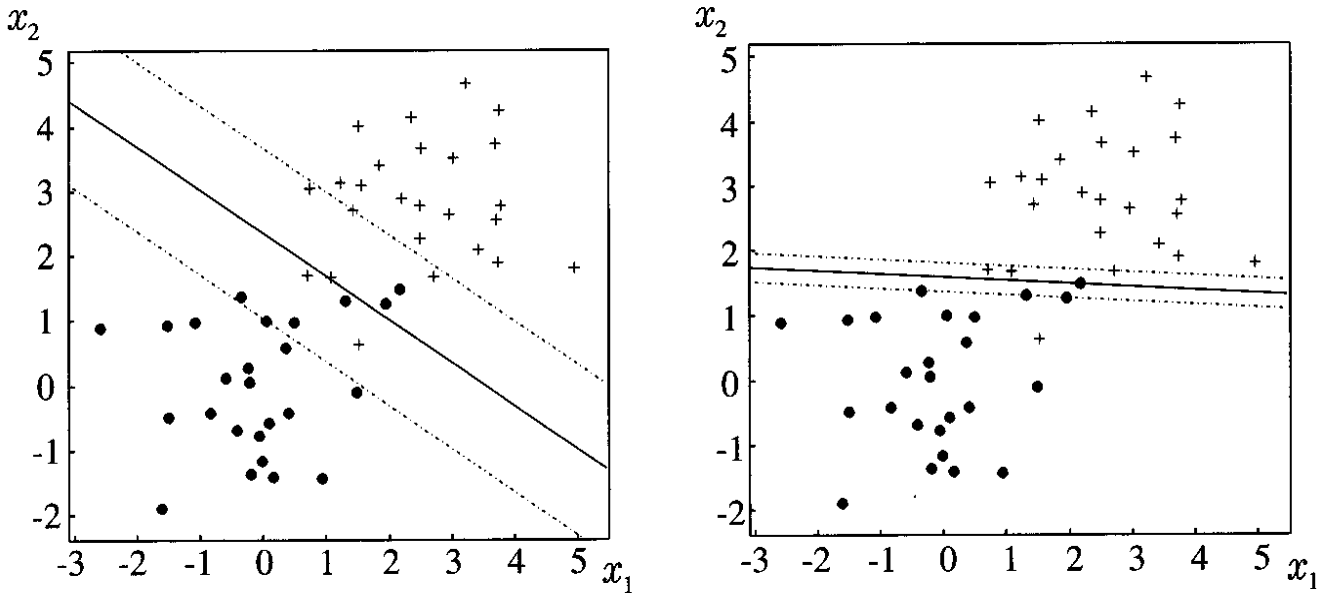
Modified cost function: $J = \frac{1}{2} \| \boldsymbol{w} \|^2 + C \sum_{i=1}^{N} \xi_i \quad (C > 0)$

Here C is an adjustable hyperparameter.

Hyperparameter C in SVM

- Hyperparameter C represents a trade-off between large margin and minimum classification error (in training data)
- C has to be selected separately (before SVM training). One common approach is to use cross-validation.
- For linearly separable cases, we can obtain the same results as obtained previously by setting $C \to \infty$.

$C = 0.2$ $\qquad$ $C = 1000$



A computer experiment result for using different C values in a nonseparable case:

Linear Metric Learning

Why Metric Learning?

Remember the metric tensor matrix $\boldsymbol{M}$ that defines an inner product, and as a result, a distance metric:

$$d_M(x_i, x_j) = \sqrt{(x_i - x_j)^T M (x_i - x_j)}$$

- Euclidean Distance ($\boldsymbol{M}$ is an identity matrix): Treats every feature axis equally and independently. It completely ignores natural scale differences, correlated features, and task-specific priorities.
- Standard Mahalanobis Distance ($\boldsymbol{M}$ is the inverse of covariance): Accounts for global variance and inter-feature covariance, but it is strictly unsupervised. It deforms space based solely on data scatter, completely ignoring class boundaries or semantic groupings.
- Real-world data requires a distance metric (or more specifically, matrix $\boldsymbol{M}$) that optimizes task performance. ➔ Obtain $\boldsymbol{M}$ through a learning (optimization) process.

Metric Learning is Supervised

- The intuitive objective: To find a distance metric such that
  - A pair of samples in the same class have a smaller distance.
  - A pair of samples in different classes have a larger distance.
- Pairwise label constraints:
  - Sample pairs in the Must-Link Set (set of sample pairs in the same class) must have small distances.
  - Sample pairs in the Cannot-Link Set (set of sample pairs in different classes) must have large distances.
- Relative (triplet) label constraints: For the distances between one anchor sample and two other samples, specify that one distance should be larger than the other.
- Other than class labels, such constraints can be derived from other forms of supervision (e.g., temporal continuity, use feedback, etc.) as well.

Properties of a Distance Metric

- Non-negativity: $d_M(x_i, x_j) \geq 0.$
- Identity of Indiscernibles: $d_M(x_i, x_j) = 0 \iff x_i = x_j$
- Symmetry: $d_M(x_i, x_j) = d_M(x_j, x_i)$
- Triangle Inequality: $d_M(x_i, x_k) \leq d_M(x_i, x_j) + d_M(x_j, x_k)$

For $d_{\boldsymbol{M}}$, the first two properties require $\boldsymbol{M}$ to be semi-positive definite.

Decomposition of Metric Tensor Matrix

$$M = L^T L$$

$$\begin{aligned} d_M(x_i, x_j) &= \sqrt{(x_i - x_j)^T (L^T L)(x_i - x_j)} \\ &= \sqrt{\left(L(x_i - x_j)\right)^T \left(L(x_i - x_j)\right)} \\ &= \sqrt{(Lx_i - Lx_j)^T (Lx_i - Lx_j)} \\ &= \|Lx_i - Lx_j\|_2 \end{aligned}$$

- Here $\boldsymbol{M}$ is dxd (d is the original feature space dimensionality)
- $\boldsymbol{L}$ is rxd (r is the rank of $\boldsymbol{L}$).
- When $r = d$: $\boldsymbol{L}$ rotates, scales, and shears the space
- When $r < d$: $\boldsymbol{L}$ compresses the high-dimensional space into a lower-dimensional subspace, simultaneously achieving dimensionality reduction.

Loss Functions for Metric Learning

The general form of loss functions using pairwise constraints:

$$\min_{M \succeq 0} \quad \text{Loss}(M, M_0) + \lambda \sum_{(i,j) \in \mathcal{S}} \text{penalty}_S(d_M(x_i, x_j)) + \mu \sum_{(i,k) \in \mathcal{D}} \text{penalty}_D(d_M(x_i, x_k))$$

must-link set

cannot-link set

- Here $\boldsymbol{M}_0$ is the prior metric matrix, often computed as inverse of global covariance.
- The first term keeps $\boldsymbol{M}$ similar to $\boldsymbol{M}_0$.
- It is also necessary to keep $\boldsymbol{M}$ positive semi-definite.
- The constraint penalty terms commonly use a distance upper bound for must-link pairs and a distance lower bound for cannot-link pairs.

Loss Functions for Metric Learning

One way to define the first term is to use the Frobenius Norm:

$$\text{Loss}(M, M_0) = \|M - M_0\|_F^2 = \sum_{i=1}^{d} \sum_{j=1}^{d} (M_{ij} - M_{0,ij})^2$$

The challenge: Optimization with this loss term does not guarantee that $\boldsymbol{M}$ will stay positive semi-definite. Post-processing of $\boldsymbol{M}$ can be computationally expensive.

Information-Theoretic Metric Learning

Information-theoretic metric learning (ITML) defines the first term as

$$\mathrm{Loss}(M, M_0) = D_{\mathrm{LD}}(M, M_0) = \mathrm{Tr}(MM_0^{-1}) - \log\det(MM_0^{-1}) - d$$

- The name comes from the fact that using the log-determinant loss between two matrices is equivalent to using the Kullback-Leibler (KL) divergence of their corresponding Gaussian distributions.
- If any eigenvalue drops toward zero, $\boldsymbol{M}$ approaches singularity, and the log-determinant loss approaches infinity. This forms a natural "barrier" that keeps $\boldsymbol{M}$ positive semi-definite without post-processing.

Margin-Based Metric Learning

- ITML and methods with similar loss function compositions attempt to find M that minimizes label constraint violation for ***all*** pair constraints.
- Margin-based metric learning methods like large-margin nearest-neighbor (LMNN) focus on ***local*** constraints, thus optimizing the metric for k-NN type classifiers.
- Two types of neighbors in LMNN:
  - Target neighbors: Neighbors of the same class as the anchor
  - Imposters: Neighbors of different classes from the anchor

Large-Margin Nearest Neighborhood

The loss function includes two terms (total loss is the weighted sum of the two terms):

:

- "Pull" terms:

$$\sum_i \sum_{j \in \eta_i} d_M^2(x_i, x_j)$$

set of target neighbors

- "Push" terms:

$$\sum_i \sum_{j \in \eta_i} \sum_k (1 - y_{ik}) \left[1 + d_M^2(x_i, x_j) - d_M^2(x_i, x_k)\right]_+$$

imposters

margin

hinge loss

The set of neighbors (targets and imposters) are usually fixed in the beginning of the optimization using, say, Euclidean distances.

Large-Margin Nearest Neighborhood

Optimization method #1: Semidefinite Programming (SDP)

$$\begin{aligned} \min_{M,\xi} \quad & (1-\mu)\sum_{i,j\in\eta_i} d_M^2(x_i,x_j) + \mu \sum_{i,j\in\eta_i,k} \xi_{ijk} \\ \text{s.t.} \quad & d_M^2(x_i,x_k) - d_M^2(x_i,x_j) \geq 1-\xi_{ijk} \quad \forall(i,j\in\eta_i,k:y_i\neq y_k) \\ & \xi_{ijk}\geq 0, \quad M\succeq 0 \end{aligned}$$

slack variables for constraint violations

Optimization method #2: Subgradient Descent

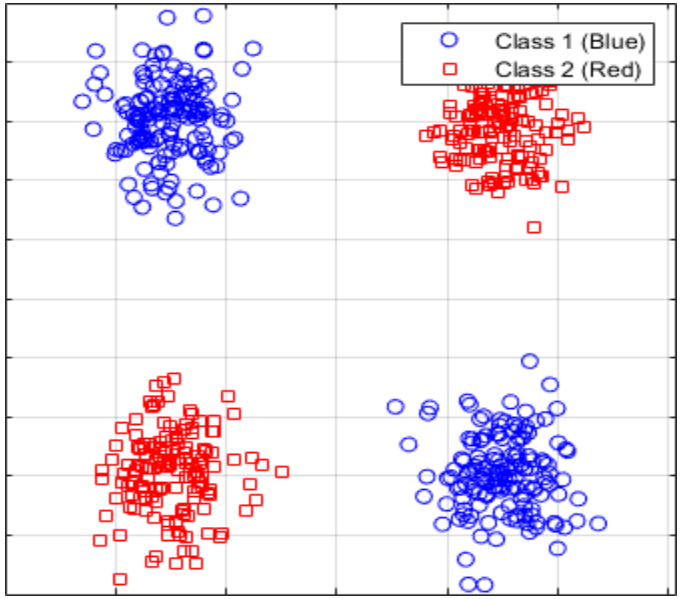
$$\frac{\partial \mathcal{E}}{\partial L} = 2L\left[(1-\mu)\sum_{i,j\in\eta_i} C_{ij} + \mu \sum_{(i,j,k)\in\mathcal{V}} (C_{ij}-C_{ik})\right]$$

all triplets that violate the margin constraint

where $C_{ab} = (x_a - x_b)(x_a - x_b)^T$

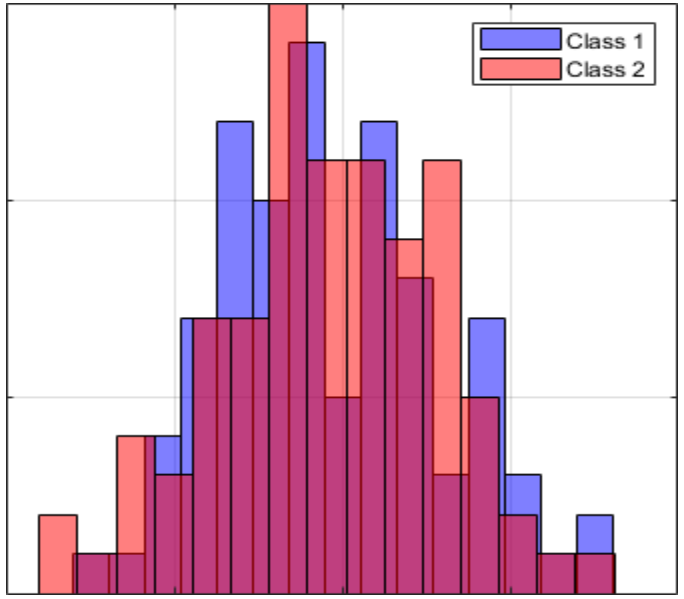
Demo: LMNN vs. LDA

When LDA fails due to overlapping global distributions of the classes, LMNN can find a metric that works for kNN classifiers.



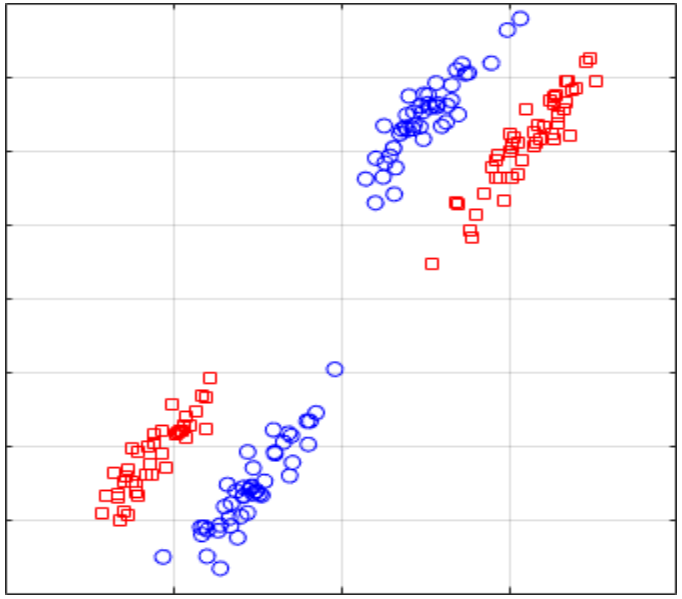
original data distribution (plus 20 noise dimensions)

1NN accuracy: 73%



distributions after 1-D projection from LDA

1NN accuracy: 44%



data distribution in metric space from LMNN

1NN accuracy: 99%

More General Metric Learning

- Kernel Metric Learning: Apply the kernel trick (to be discussed later) so $\boldsymbol{M}$ is learned in an implicit high-dimensional space.
- Local / Multi-Metric Learning: Learns multiple local matrices for different regions or clusters of feature space instead of one global matrix $\boldsymbol{M}$.
- Deep Metric Learning: Replaces the linear projection matrix $\boldsymbol{L}$ with a non-linear deep neural network mapping to an embedding space where classes can be more linearly separable.